# The Claude Code Leak: 10 Agentic AI Harnes Patterns That Change Everything

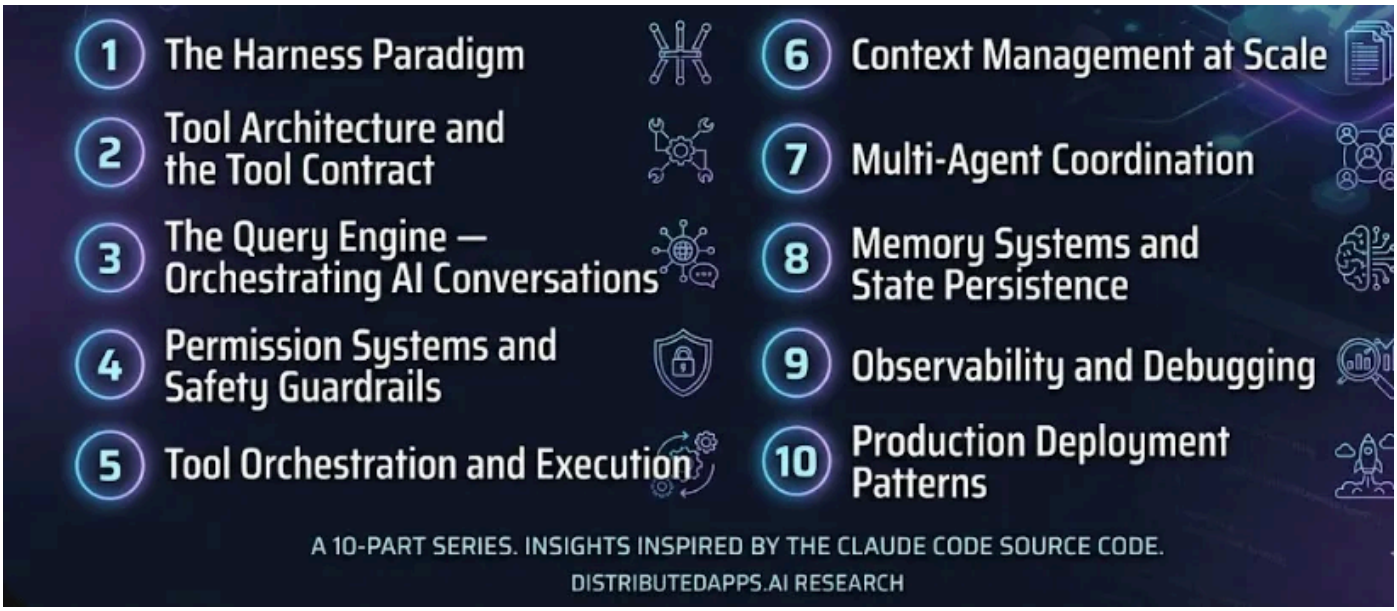
KEN HUANG
APR 01, 2026

51 2 Share

Following the recent leak of Anthropic's Claude Code source code, our research tea at DistributedApps.ai performed a deep dive into the repository to decode the agen AI engineering patterns hidden within its vast architecture.

We identified **10 fundamental patterns** that serve as the backbone for production-ready agentic systems. These are not merely specific to Anthropic; they are univers principles essential for any team building at the frontier of AI.

Starting tomorrow, we are launching a **10-part series** exploring each pattern in deta featuring direct code examples from the codebase. Here is a preview of what's ahea



# Chapter 1: The Harness Paradigm

The most fundamental insight from the Claude Code codebase is that the challenge production AI is not about the model — it is about the harness. The model provides intelligence, but the harness provides control. A well-designed harness can make ev a modest model reliable and safe, while a poorly designed harness can make even th most capable model dangerous.

The harness addresses five core challenges: constraining the action space through tools, managing conversation state, enforcing safety through permissions, handling failures gracefully, and optimizing resource usage. The three-layer architecture — Model Layer, Harness Layer, and UI Layer — separates concerns and enables clean system design. The Tool interface is the fundamental abstraction that enables the harness to reason about tool behavior and make intelligent decisions.

**In the full series**: We examine the harness paradigm in detail, showing how the thr layer architecture separates concerns, how the Tool interface creates a uniform contract for all actions, and how the core challenges of safety, reliability, and scalability are addressed through permissions, compaction, and multi-agent coordination.

# Chapter 2: Tool Architecture and the Tool Contract

Every tool in the Claude Code system implements the same interface, creating a uniform contract between the model and every possible action it can take. This interface covers identity, execution, validation, permissions, and presentation. The insight is that the harness must be able to reason about tool behavior — whether to can run concurrently, whether they modify state, whether they require special permissions — and make intelligent decisions without understanding the tool's internals.

Zod schemas provide type-safe validation at every boundary. Behavioral properties isConcurrencySafe and isReadOnly enable optimization. The buildTool factory provides conservative defaults that make new tools safe out of the box.

**In the full series**: We walk through the complete Tool interface, explaining each fie and showing how they work together to create a tool system that is both expressive and safe.

## Chapter 3: The Query Engine — Orchestrating AI Conversations

The QueryEngine is the central orchestrator of the AI conversation lifecycle. It manages the back-and-forth between the user, the language model, and the tool execution environment. The query loop implements what we call the "continue site pattern" — multiple exit points that can either terminate the turn or continue to th next iteration with updated state, enabling sophisticated error recovery.

The harness withholds recoverable errors and attempts recovery through multiple strategies: context collapse drain, reactive compact, max output tokens escalation, a multi-turn continuation. Token budgets are enforced at multiple levels to prevent runaway costs.

**In the full series**: We trace the complete query loop, identifying all continue sites a showing how they enable graceful degradation and self-healing behavior.

## Chapter 4: Permission Systems and Safety Guardrails

The permission system in Claude Code is not a single check but a layered pipeline. First, general permission rules are evaluated — allow lists, deny lists, ask lists. Ther tool-specific checks run, allowing tools to implement custom logic. Then automate

classifiers can make fast-path decisions. Finally, if no automated path resolves the decision, the system falls back to interactive user approval.

The system operates in multiple modes — default, auto, plan, acceptEdits, bubble – each representing a different balance between automation and user control. The canUseTool function is the central gatekeeper, ensuring that every tool use is evaluated before execution.

**In the full series**: We examine the canUseTool function and trace a permission evaluation through all layers, showing how the system makes intelligent decisions without frustrating users.

## Chapter 5: Tool Orchestration and Execution

When the model produces multiple tool_use blocks in a single response, the harnes must decide execution order. The Claude Code system partitions tools into batches based on concurrency safety. Consecutive concurrency-safe tools are batched togetl and executed in parallel. Non-safe tools run serially, each in its own batch.

The StreamingToolExecutor processes tools as they arrive during streaming, yieldi results in real time. Progress tracking enables users to see what is happening as too execute. When tools fail, the harness generates synthetic tool_result blocks to maintain the invariant that every tool_use has a matching tool_result.

**In the full series**: We show the partitioning algorithm, the concurrent execution us async generators, and how context modifiers are queued and applied after batch completion.

## Chapter 6: Context Management at Scale

The Claude Code system implements multiple context management strategies that work together in a hierarchy. Snip compaction removes specific message ranges.

Micro-compact edits cached prompt cache entries. Context collapse archives old messages and projects a collapsed view. Auto-compact summarizes old conversatio history. Reactive compact catches overflow errors as a fallback.

These strategies are applied in order of cost — cheap local operations first, expensi API calls only when necessary. The system tracks token counts and triggers compaction at specific thresholds, with a circuit breaker that stops retrying after consecutive failures.

**In the full series**: We explain each compaction strategy, show the threshold management system, and trace how the query loop applies them in order.

## Chapter 7: Multi-Agent Coordination

The AgentTool is the primary mechanism for spawning and coordinating multiple agents. It supports several execution modes: synchronous subagents that block unti completion, asynchronous agents that run in background, fork subagents that inhe the parent's context, teammates that can be addressed by name, and remote agents that execute in separate environments.

The harness manages the full lifecycle of spawned agents — allocating resources, tracking progress, handling errors, and cleaning up on completion. Worktree isolat provides filesystem separation between agents. System prompt inheritance allows f subagents to share the parent's prompt cache, reducing API costs.

**In the full series**: We show how fork subagents are spawned, how system prompt inheritance works, and how the parent's conversation history is packaged for the ch

## Chapter 8: Memory Systems and State Persistence

The harness maintains persistent conversation state across turns and sessions. The QueryEngine's mutableMessages array holds the complete conversation history. Transcript recording persists conversations to disk for recovery after crashes. The f state cache avoids redundant file reads. Memory attachments are automatically injected when relevant, with deduplication to prevent re-injection.

Session memory compaction preserves the memory directory while summarizing conversation history. Preserved segments after compaction ensure that resume wor correctly. Agent metadata is written to disk for recovery. Skill discovery tracking records which skills were surfaced during a session.

**In the full series**: We trace the memory prefetch lifecycle from initiation to consumption, showing how it integrates with the query loop.

## Chapter 9: Observability and Debugging

Every significant event in the Claude Code system is logged with structured metada Agent spawns, compactions, errors, permission decisions, model fallbacks — all are captured with rich context. A branded type ensures analytics metadata never contai code or filepaths, preventing accidental PII leakage. Query chain IDs trace conversations across turns and subagent spawns.

Headless profiling checkpoints measure latency at key points in the query lifecycle. The query profiler tracks timing for snip, micro-compact, and auto-compact operations. Error diagnostics provide rich context when things go wrong, including the exact reason why each error fired.

**In the full series**: We catalog the key events, show the analytics metadata type syste and demonstrate how query chain tracking enables end-to-end tracing.

## Chapter 10: Production Deployment Patterns

The harness is designed to be consumed via an SDK interface. The QueryEngine exposes an async generator that SDK consumers can iterate. SDK message types co all event types that consumers need to handle. Feature flags control which experimental features are enabled, with thirty flags covering everything from autonomous agent modes to remote execution environments.

The harness supports multiple LLM providers through an abstraction layer — Anthropic Direct, AWS Bedrock, Google Vertex, and Azure Foundry. Permission modes are configurable for different deployment scenarios. The plugin system exte functionality, and MCP integration connects to external tools and resources.

**In the full series**: We show the SDK integration patterns, the feature flag system, a how multi-provider support is implemented.

## Coming Soon: The Full 10-Part Series

These ten patterns represent the distilled wisdom of building production AI agent systems at scale. They are not theoretical — they have been tested with millions of agent interactions. They are not proprietary — the principles apply regardless of which LLM you use. And they are not optional — any team building serious AI age needs to understand them.

Starting tomorrow, we will publish one deep-dive article per pattern, each with full code examples from the Claude Code codebase. You will see exactly how each patte is implemented, why the design decisions were made, and how you can apply them your own systems.

**Subscribe now** to make sure you do not miss any installment. Each article will be published exclusively on Substack, and subscribers will receive them directly in the inbox.

---

## Subscribe to Agentic AI

By Ken Huang · Hundreds of paid subscribers

I share my thoughts on anything related to Agentic AI and Agentic AI Security topics.

Upgrade to paid

51 Likes · 2 Restacks

← Previous Next

## Discussion about this post

Comments Restacks

Write a comment...

© 2026 ken · Privacy · Terms · Collection notice
Substack is the home for great culture